Building Tools for AnyLM

A practical guide to giving local models hands: tool calling, custom tools, and access to other applications.

AnyLM can let the model do more than answer. When tools are enabled for a chat, the model can read files, call APIs, launch applications, and run commands on your computer, then use the results to continue the conversation. This guide explains how the tool system works, how to build your own tools without writing app code, and how to extend AnyLM with new built-ins.

How it works

AnyLM uses Ollama's function calling. Each enabled tool is described to the model as a function with a name, a description, and typed parameters. The flow for one turn looks like this:

1. You send a message with the Tools toggle on. AnyLM attaches every enabled tool's schema to the request.
2. The model either answers normally or emits a **tool call**: the tool's name plus arguments.
3. AnyLM executes the tool (asking you first if it's risky) and appends the output to the conversation.
4. The model is invoked again with the result and continues, up to five tool rounds per turn.

Everything happens locally: the model runs in Ollama, tools run in the AnyLM desktop app, and governance still applies — token metering, policy filters, and compliance logging all see tool-assisted turns like any other.

> Note: your model must support function calling (e.g. llama3.1/3.2, qwen2.5, mistral-nemo, firefunction). Models without tool support simply answer normally.

Built-in tools

Tool	What it does	Risky?
get_time	Current date and time	No
read_file	Read a text file (up to 512 KB)	No
list_directory	List files and folders in a directory	No
open_app_or_url	Open an application, file, or URL	Yes
run_shell	Run a shell command and return its output	Yes
http_fetch	Call an HTTP API and return the response	GET: no; other methods: yes

Manage these in the **Tools** view in the sidebar. Toggle any tool off and the model will never see it.

The safety model

Tools are only offered to the model when you switch on the Tools toggle in a chat's composer, per conversation. Read-only tools (time, files, directory listings, HTTP GET) run automatically. **Risky tools** — shell commands, app launches, and non-GET HTTP requests — always show a confirmation dialog with the exact command or arguments before anything runs. If you don't respond within two minutes, the call is denied automatically, and the model is told you declined.

Building a custom tool

Open **Tools** → **New tool**. A custom tool is a small recipe with five parts:

Field	Meaning
Name	snake_case identifier the model calls, e.g. search_notes
Description	One sentence telling the model when to use it. This matters most.
Kind	Shell command or HTTP request
Command / URL	What to execute, with {param} placeholders
Parameters	One per line: name | description | required

Parameter values supplied by the model are substituted into {param} placeholders. Shell values are automatically quoted and escaped; URL values are URL-encoded. Output is captured (stdout + stderr, truncated to 20 KB) and handed back to the model.

Example 1 — search your notes folder (shell)

```
Name:        search_notes
Description: Search the user's notes folder for a phrase and return matching lines.
Kind:        Shell command
Command:     grep -ri {query} ~/Documents/Notes --include='*.md' -l -m 20
Parameters:  query | The phrase to search for | required
```

Ask the model "where did I write about the Q3 roadmap?" and it will call search_notes(query="Q3 roadmap") and answer from the results.

Example 2 — check a local service (HTTP)

```
Name:        homeassistant_state
Description: Get the current state of a Home Assistant entity (lights, sensors, switches).
Kind:        HTTP request      Method: GET
URL:         http://homeassistant.local:8123/api/states/{entity_id}
Parameters:  entity_id | Entity such as light.kitchen or sensor.temperature | required
```

Example 3 — create a calendar event (shell + AppleScript, macOS)

```
Name:        add_calendar_event
Description: Create a calendar event today with a title and 24h start time like 15:00.
Kind:        Shell command
Command:     osascript -e 'tell application "Calendar" to tell calendar "Home" to make new event
             with properties {summary:'{title}', start date:(current date) + (({hour}) * hours)}'
Parameters:  title | Event title | required
             hour  | Hours from midnight, e.g. 15 for 3pm | required
```

Recipes: reaching other applications

The shell is the universal adapter. Almost every application on your computer exposes some command-line or scripting surface; a custom shell tool turns it into a model capability.

macOS

```
# Open an app                    open -a "Spotify"
# Play/pause Spotify             osascript -e 'tell application "Spotify" to playpause'
# Send an iMessage               osascript -e 'tell application "Messages" to send {text}
                                 to buddy {recipient}'
# Create a note                  osascript -e 'tell application "Notes" to make new note
                                 with properties {body:{text}}'
# System notification            osascript -e 'display notification {text}'
# Reveal a file in Finder        open -R {path}
```

Windows

```
# Open an app                    start "" "spotify"
# Speak text aloud               powershell -c "Add-Type -AssemblyName System.Speech;
                                 (New-Object System.Speech.Synthesis.SpeechSynthesizer).Speak({t
# Create a file in Notepad       powershell -c "Set-Content -Path {path} -Value {text}; notepad
```

Linux

```
# Open an app / URL              xdg-open {target}
# Desktop notification           notify-send {title} {body}
# Clipboard                      echo {text} | xclip -selection clipboard
```

HTTP-native applications

Anything with a local or cloud REST API works with an HTTP tool: Home Assistant, Jira, Notion, Obsidian (Local REST API plugin), Jellyfin, qBittorrent, printers, routers. For services that need an API key, put the key directly in the URL template or wrap the call in a small shell script that adds headers with curl:

```
Name:        jira_search
Kind:        Shell command
Command:     curl -s -H "Authorization: Bearer $JIRA_TOKEN"
             'https://jira.internal/rest/api/2/search?jql={jql}&maxResults=5'
Parameters:  jql | A JQL query like 'assignee=me AND status="In Progress"' | required
```

Writing descriptions the model understands

The description is the model's only clue about when to use a tool. Good descriptions state the action, the domain, and the expected input. Compare:

Weak	Strong
"notes tool"	"Search the user's markdown notes folder for a phrase; returns matching file paths."
"spotify"	"Play a specific Spotify playlist by its playlist ID. Use when the user asks for music."

Weak	Strong
"api call"	"Get today's calendar events as JSON. Use before answering questions about the user's schedu

Parameter descriptions matter too — "24h start time like 15:00" prevents the model from sending "3pm". If the model keeps misusing a tool, sharpen these sentences before anything else.

Security practices

• **Prefer narrow tools over run_shell.** A tool that can only grep one folder is far safer than a general shell. Disable run_shell in the Tools view if you don't need it.
• **Keep risky confirmation on.** The confirmation dialog shows the exact substituted command; read it before allowing.
• **Parameters are escaped, commands are not.** The {param} values are shell-quoted, but the command template itself runs verbatim — never paste untrusted text into a template.
• **Mind prompt injection.** Documents and web pages the model reads can contain instructions that try to trigger tools. Risky-tool confirmation is your backstop; keep it enabled when attaching untrusted files.
• **Timeouts and caps.** Shell tools stop after 15 seconds; outputs are truncated to 20 KB; at most five tool rounds run per message.

For developers: extending AnyLM itself

Tool code lives in two small modules of the Electron main process:

File	Responsibility
app/src/main/tools/registry.js	Tool definitions: built-ins, custom tool storage, Ollama schema export
app/src/main/tools/exec.js	Execution: built-in implementations, shell/HTTP runners, risk gating
app/src/main/ipc.js	The agent loop inside chat:start (stream → tool calls → execute → re-invoke)
app/src/renderer/js/tools-view.js	Tools manager UI and the custom tool editor

Adding a built-in takes two steps. First declare it in registry.js:

```
// registry.js — BUILTINS
{
  id: "clipboard_read",
  name: "clipboard_read",
  builtin: true,
  risky: false,
  description: "Read the current text contents of the clipboard.",
  params: [],
}
```

Then implement it in exec.js:

```
// exec.js — execBuiltin switch
case "clipboard_read": {
  const { clipboard } = require("electron");
  return clip(clipboard.readText() || "(clipboard is empty)");
}
```

Set `risky: true` for anything that writes, sends, deletes, or spends — the confirmation flow picks it up automatically. Tool results should always be strings; return "Error: …" rather than throwing so the model can recover and explain.

Tools and the /v1 API

> Tool execution is a desktop-app capability. Requests routed through the OpenAI-compatible /v1/chat/completions proxy (API keys) are governed and metered but do not execute AnyLM tools; external apps are expected to run their own. If you need shared tool execution, treat it as a roadmap item: move exec.js behind the backend and gate it with a tool-allowlist policy.

Quick checklist

1. Enable the built-ins you want in **Tools**; disable the rest.
2. Create custom tools for the two or three apps you actually use daily.
3. Write one-sentence descriptions that say when to use the tool.
4. Flip the Tools toggle on in a chat and ask for something concrete.
5. Watch the inline tool notes to see what ran; refine descriptions when the model guesses wrong.